Bumper Brawl: 4-Person Build Plan

Oct 24, 2026 · @Raj

How the split works

All four people start real work at 16:30, because every hand-off is replaced by a contract frozen in the first 45 minutes and a stand-in that fakes the missing part.

- **Frozen contracts:** message names and shapes, every tunable number, the mechanics function signatures, and sample snapshot JSON. Code is written against these, not against each other's code.
- **Stand-ins:** the server is tested with bots instead of phones. The phone screens are tested on a mock network that plays fake snapshots. The rules are pure functions tested with no server. The server starts on stub rules that return safe defaults.
- **One owner per file:** only the owner edits a file. Changes to anything in `/shared` need a quick ping to the team first.

Person	Role	Owns
1	Server core	`server/index.js`, `rooms.js`, `game.js`, `bots.js`, a `/debug` page
2	Phone controls and drawing	`client/src/input.js`, `render.js`, `interp.js`, `screens/game.js`
3	Rules and tuning	`server/mechanics.js` and its tests, `shared/config.js`, `constants.js`, `server/sim.js`, results screen, sounds
4	Deploy, network, screens, submission	VM and `deploy.sh`, `client/src/net.js`, `mocknet.js`, `main.js`, home, lobby and spectate screens, Devpost, video, pitch

Persons 1 and 2 have the heaviest builds. Person 3 shifts to tuning and playtesting once the rules are done. Person 4 carries the submission on Sunday morning.

Hour zero: 15:45 to 16:30, all four together

Nobody splits off until these files are pushed, because every later task is written against them.

1. **Repo skeleton** (Person 1 types). Use the folders from the design doc. `npm run dev` runs Vite plus `node --watch server/index.js`.
2. **`shared/protocol.js`** (everyone). Use the design doc's events, with two changes:
   - `input` becomes `{ seq, tilt, accel, taps, shakes, jerks }`. The last three are counts since the previous message.
   - New event `spectate { code }`. Its ack is `{ ok }`, after which the spectator receives `lobby`, `snap` and `events` like a player.
3. **`shared/config.js` and `constants.js`** (Person 3 types). Include every tunable, plus the new ones in the next section.
4. **`server/mechanics.js` stubs** (Person 3 types, Person 1 checks). These are the signatures below with simple bodies: drive forward when accelerating, turn at `tilt * turnRate`, and never deal damage. Person 1 can run the server on these immediately.
5. **Fixtures** (Person 4 types). `shared/fixtures/lobby.json`, `snap.json` (4 cars, one flipped, one dead) and `events.json` (one of each event type).
6. **Conventions.** The world is 1000 × 600 units, origin top-left, y pointing down. Angles are radians, 0 faces +x, and forward is `(cos a, sin a)`. Physics runs in meters (units ÷ 20). Snapshots use world units.

```
// server/mechanics.js: pure functions, plain numbers in and out

driveStep(car, input, preset, cfg)
// car: { vx, vy, angle, w, mass, state }   (m/s, rad, rad/s)
// returns { lateralImpulse:{x,y}, force:{x,y}, minForwardSpeed,
//           angularVelocity, linearDamping }

resolveHit(a, b, nx, ny, now, cfg)
// a, b: { id, vx, vy, angle, mass, state, parryUntil, preset }
//       captured in begin-contact, before the collision is solved
// returns { dmgToA, dmgToB, flipA, flipB, parriedBy }

wallHit(car, nx, ny, cfg)        // returns { speedKeep, stop, selfDmg }
flipTick(car, input, dtMs, cfg)  // returns { state, recoverProgress,
invulnMs }
scoreRound(players, winnerTeam)  // returns leaderboard[]
```

Design fixes folded into this plan

These replace the matching parts of the design doc. Each fix has an owner, so it ships as part of normal work.

Problem in the design doc	Fix	Owner
Full-lock turns flip balanced and light cars, and collisions can't spin anyone	Steering eases toward its target: `w += (target − w) × STEER_BLEND`. Flips come only from side hits at `FLIP_HIT_SPEED` or more, or from hit-caused spin above `FLIP_SPIN` for 250 ms. Delete `FLIP_LOAD`.	3 writes the rules, 1 applies them
Damage is computed after the bounce, so it comes out near zero	Record velocities, angles and the contact normal in `begin-contact`. Queue them and call `resolveHit` after `world.step`.	1 captures, 3 computes
A full-speed hit kills in one shot	`DMG_K` drops to 0.2, about 40 damage at 14 m/s	3, checked in unit tests
The spectator page can't join a room	New `spectate` event	1 server side, 4 page
Unclear snapshot rate, and timers drift	Fixed-step accumulator for physics at 30 Hz. A separate 50 ms accumulator for snapshots.	1
Cars jitter with only 2 snapshots buffered	Interpolation delay 150 ms	2
`screen.orientation` is missing before iOS 16.4	Fall back to `window.orientation`	2
Tilt alone is the obvious use of phone motion	Shake to recover from a flip, a quick jerk to parry. Taps still work as a fallback for both.	2 detects, 3 rules
Devpost closes at noon Sunday, but the design doc froze features at noon	Freeze at 09:00, record the video at 10:00, submit by 11:15	4

```
// shared/config.js: new or changed values
STEER_BLEND: 0.3,
DMG_K: 0.2,
FLIP_HIT_SPEED: 9,      // m/s into the victim's side
FLIP_SIDE_MIN: 0.6,     // 0 = head-on, 1 = square T-bone
FLIP_SPIN: 7,           // rad/s, above every preset's turn rate
FLIP_HOLD_MS: 250,
RECOVER_SHAKES: 12,     // shakes OR taps, whichever finishes first
RECOVER_TAPS: 10,
SHAKE_ACCEL: 14,        // m/s² without gravity
SHAKE_GAP_MS: 120,
JERK_ACCEL: 22,         // parry gesture
SNAP_EVERY_MS: 50,
INTERP_DELAY_MS: 150,
```

Person 1: server core

Person 1 builds the game server and tests it alone with bots and a 20-line `/debug` page, so no phone is needed until 20:00.

Build alone (16:30 to 20:00)

- [ ] `index.js`: Express serves the client build, Socket.IO on the same port, a `/health` route. Push by 16:45 so Person 4 can deploy it.
- [ ] `rooms.js`: join codes, create, join, team, car, mode, ready, start, the lobby → countdown → playing → roundEnd → lobby states, host handoff, and `spectate`.
- [ ] `game.js`: Planck world, walls, pillars, spawns. Physics on a fixed-step accumulator at 30 Hz. Apply `driveStep` results (stubs at first).
- [ ] `game.js` contacts: in `begin-contact`, copy both cars' velocity, angle, mass and state, plus the normal, into a queue. After `world.step`, call `resolveHit` on each item.
- [ ] `game.js` snapshots: every 50 ms, rounded as the design doc says, with that tick's events batched.
- [ ] `bots.js`: `?bots=3` adds cars that chase the nearest enemy at full throttle.
- [ ] `/debug` page: draws each car as a rectangle from `snap`. It's only for you.

Integrate (20:00 to 02:00)

- [ ] 20:00: one real phone drives one car, with Person 2.
- [ ] Swap the stubs for Person 3's real rules as soon as they're pushed (target 18:30).

- [ ] Deaths, win check, `roundEnd`, and the 5-second disconnect rule.
- [ ] Flipped state: zero velocity, heavy damping, recovery from `shakes` or `taps`.
- [ ] Lobby events live for Person 4 by 19:30.

Harden (Sunday 07:00 to 09:00)

- [ ] Parry window, if Person 3 has the rule ready.
- [ ] Wrap the tick in try/catch so one bad contact can't kill the room. Clean up empty rooms.
- [ ] Load test: 8 players, real phones plus bots, on the hotspot.

Person 2: phone controls and drawing

Person 2 makes the phone feel good to drive and draws the arena. They work from the fixture JSON first, then Person 4's mock network, then the real server.

Build alone (16:30 to 20:00)

- [ ] A one-file tilt test page that prints raw gravity values and the steering output. Person 4 hosts it on HTTPS by 16:45. Confirm the axis and sign on an iPhone and an Android phone by 17:30, then write the numbers into config.
- [ ] `input.js`: motion permission on a user tap, gravity steering, calibration, deadzone, `window.orientation` fallback.
- [ ] `input.js` buttons: hold ACCEL on the right half, TAP on the left half, using pointer events.
- [ ] `input.js` gestures: count a shake when acceleration without gravity passes `SHAKE_ACCEL`, at most one per `SHAKE_GAP_MS`. Count a jerk above `JERK_ACCEL` while alive. Send `input` at 30 Hz with counts since the last message.
- [ ] Export `calibrate()` so the lobby's Recalibrate button can call it.
- [ ] `render.js`: letterbox, devicePixelRatio, arena, pillars, cars in team colors, HP bars, own car outlined. Draw `snap.json` first.
- [ ] `interp.js`: keep 10 snapshots, draw 150 ms behind, take the shortest path for angles.

Integrate (20:00 to 02:00)

- [ ] 20:00: one real phone drives one car, with Person 1.
- [ ] Effects from events: hit sparks, damage numbers, screen shake on the victim, shield ring, and a "FLIPPED! SHAKE!" overlay with a counter.

- [ ] `screens/game.js`: countdown that calibrates, rotate-your-phone overlay, wake lock, and spectate mode after dying.
- [ ] Tune steering feel (`TILT_MAX_G`, deadzone, shake thresholds) with Person 3 on real phones.

Polish (Sunday 07:00 to 09:00)

- [ ] Check frame rate on the oldest phone the team has. Cut effects before cutting interpolation.

Person 3: rules and tuning

Person 3 writes every game rule as a pure function with unit tests, so nothing they do waits on the server. Later they own how the game feels.

Build alone (16:30 to 20:00)

- [ ] `mechanics.js`: `driveStep` with the steering blend, grip, launch speed and coasting.
- [ ] `mechanics.js`: `resolveHit` with directional, speed-squared damage, mass ratio, minimum impact speed, the 300 ms per-pair cooldown, extra damage on flipped cars, and flips from side hits.
- [ ] `mechanics.js`: `wallHit`, `flipTick` (recover by shakes or taps, 1 s invulnerability), `scoreRound`, and the parry rule.
- [ ] `mechanics.test.js` with `node:test`. Check that head-on hits split damage about in half, getting rear-ended is the worst case, taps below 2 m/s do nothing, a full-speed side hit does about 40, and full-lock steering never flips any preset.
- [ ] Push the real rules by 18:30 so Person 1 can swap out the stubs.
- [ ] Fill in `constants.js` presets and team colors.

Integrate (20:00 to 02:00)

- [ ] `sim.js`: runs 8 bots for 50 rounds with no screen and prints average round length, hits per kill and flips per minute. Targets: rounds of 60 to 90 s, 2 to 3 hits per kill, a flip roughly every 20 s.
- [ ] Tuning on real phones from 23:00. Change numbers only in `config.js`.
- [ ] `screens/results.js`: winner and leaderboard. Also `sfx.js` with hit, flip and kill sounds.

Playtest (Sunday 07:00 to 09:00)

- [ ] Hand phones to hackers who have never played. Loosen anything that confuses them, especially flips.

Person 4: deploy, network, screens, submission

Person 4 gets HTTPS live first, because iPhone motion only works over HTTPS. Then they build the fake network that unblocks Person 2, the menus, and on Sunday the submission.

Build alone (16:30 to 20:00)

- [ ] VM with Node 20, pm2 and Caddy on an `sslip.io` hostname. A hello-world page on HTTPS by 16:45, plus Person 2's tilt test page.
- [ ] `deploy.sh`, and a two-line note on how anyone runs it.
- [ ] `net.js` by 17:00: `on(event, fn)` and `emit(event, payload, ack)` wrapping Socket.IO. Push the interface first so Person 2 can code against it.
- [ ] `mocknet.js` by 17:45, turned on with `?mock=1`. It plays a fake lobby, a countdown, 4 cars driving in circles at 20 Hz, random hit, flip and kill events, and a `roundEnd`, all built from the fixtures.
- [ ] `main.js` router, home screen (name, create, join), and `manifest.json`.
- [ ] Lobby: room code, QR to `/join/CODE`, player list, team and car pickers, ready toggle, mode and Start for the host, Recalibrate button.
- [ ] Spectate page: `render.js` with the HUD off, live leaderboard, kill feed, join QR.

Integrate (20:00 to 02:00)

- [ ] Switch the lobby from the mock to the real server once Person 1's events are live (target 19:30).
- [ ] Run a full round with every phone on a teammate's hotspot, not venue Wi-Fi.

Submit (Sunday 08:00 to 11:15)

- [ ] Devpost draft from 08:00: what it is, how the three Mystery Stack ingredients are used, how it's built, what's next.
- [ ] Record the video (5 minutes max) from 10:00. Include who you are, why you built it, how it works, and future plans.
- [ ] Submit by 11:15 with a public GitHub link, all four members, and the Mystery Stack and Grand Prize tracks selected (add the GoDaddy track only if you registered a domain there). Edits stay open until 13:45.
- [ ] Pitch script, printed QR codes, phones charged.

Who could wait on whom

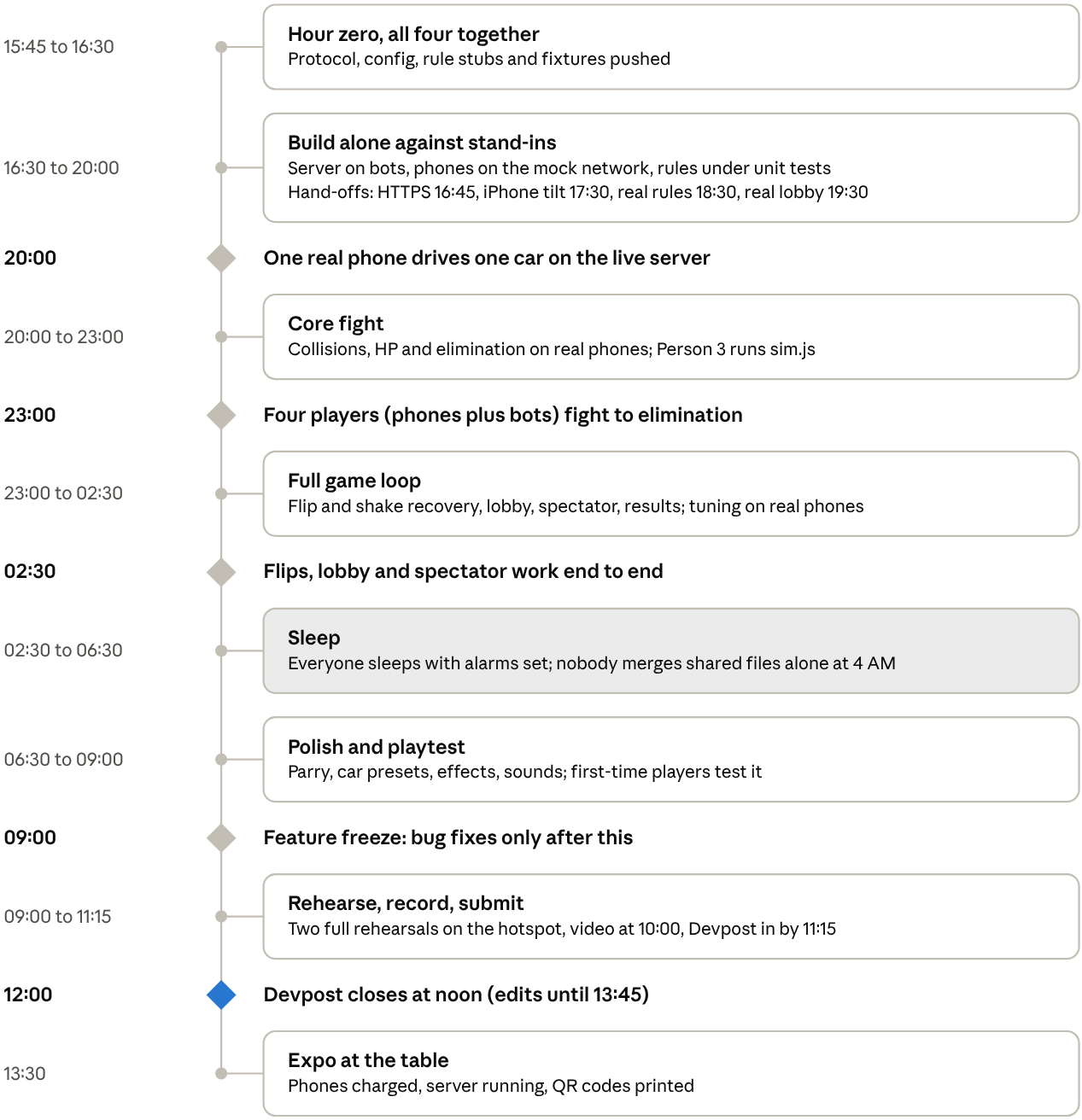
No wait is longer than about 15 minutes, because each one has a stand-in ready from hour zero.

Who waits	For what	From	Ready by	Meanwhile, use
Everyone	Protocol, config, signatures, fixtures	All four	16:30	Nothing. Do hour zero together.
2	An HTTPS page to test tilt on an iPhone	4	16:45	Write `input.js` logic, test on Android over local Wi-Fi
2	`net.js` interface	4	17:00	A 5-line inline stub with the same `on` / `emit`
2, 4	Moving snapshots	4 (mock) then 1 (real)	17:45 / 20:00	Static `snap.json`
1	Real rules	3	18:30	Stub `mechanics.js` from hour zero
4	Real lobby events	1	19:30	`mocknet.js`
3	A running game loop for `sim.js`	1	20:00	Unit tests
1	Real phone input	2	20:00	Bots and the `/debug` page
3	Real phones for tuning	1, 2, 4	23:00	`sim.js` numbers
4	A working game to film	Everyone	Sun 09:00	Devpost text and pitch script

Timeline and checkpoints

A playable fight by 23:00 and a submission by 11:15 Sunday

Saturday Oct 24 to Sunday Oct 25, Eastern time



build timeline · 8 phases, 5 checkpoints

Each checkpoint is a short demo at a standup. If one slips by more than an hour, cut features in the design doc's order: sound, parry, car presets, kill feed. Never cut flips.

Git and communication rules

File ownership keeps merge conflicts near zero. These rules keep the contracts trustworthy.

- Everyone works on `main`. Pull before every push, and commit small every 30 to 60 minutes.
- Edit only files you own. For `/shared`, post what changed in the team chat before pushing.
- Protocol changes only add fields. Never rename or remove one while others are coding against it.
- `npm test` must pass before anyone pushes `mechanics.js` or `config.js`.
- Anyone can run `deploy.sh` after pushing. Post "deployed" with the commit so testers reload.
- Two-minute standups at 18:00, 20:00, 23:00, 02:00 and 08:00: what's done, what's next, what's blocking you.
- Blocked for more than 15 minutes means say so right away, and switch to the stand-in in the table above.